

# KIMVIEWare

Plateforme générique à microservices pour le test logiciel

---

## RAPPORT DE PROJET UE PROJET

**Auteur :** []

**Encadrant :** Pr. DJAM Xaveria Youh KIMBI

**Contribution :** Extension de la Phase 1 – Détection JavaScript et extraction symbolique avec Arcon

2 mai 2026

# 1 Introduction

## 1.1 Contexte et motivation

Le test logiciel est depuis longtemps reconnu comme l'une des activités les plus critiques du cycle de développement logiciel. Il garantit que les systèmes répondent aux exigences fonctionnelles et non fonctionnelles avant leur mise en production. Avec l'essor rapide des applications à grande échelle, distribuées et intensives en données, le coût et la complexité des tests ont considérablement augmenté. Des études récentes estiment que les activités de test peuvent consommer plus de la moitié de l'effort total de développement.

Ce coût élevé est aggravé par trois défis techniques persistants : la génération de cas de tests redondants, les limitations de passage à l'échelle de l'exécution symbolique, et le décalage persistant entre la recherche académique et la pratique industrielle.

L'un des problèmes les plus étudiés est celui de la redondance dans la génération automatique de tests. De nombreuses approches produisent des cas de tests qui se chevauchent ou sont quasi-identiques, augmentant le temps d'exécution sans contribuer significativement à la couverture ou à la détection de défauts.

Parallèlement, l'exécution symbolique est apparue comme une technique puissante pour explorer systématiquement les chemins d'un programme. Cependant, son efficacité est limitée par le problème bien connu de l'explosion des chemins (*Path Explosion Problem*) : le nombre de chemins d'exécution réalisables croît de manière exponentielle avec la taille du programme.

Pour surmonter ces limitations, plusieurs approches hybrides ont été proposées. Le test logiciel basé sur la recherche (*Search-Based Software Testing*, SBST), notamment via les algorithmes génétiques, permet d'optimiser la génération de tests en équilibrant couverture et diversité. De même, les méthodes hybrides combinant exécution symbolique et heuristiques ont montré un potentiel significatif.

Cependant, ces solutions restent fragmentées et rarement intégrées dans des frameworks extensibles. Les architectures à microservices offrent une piste intéressante pour modulariser les plateformes de test et améliorer leur scalabilité.

Malgré ces avancées, un fossé persistant entre la recherche académique et la pratique industrielle demeure. De nombreux outils ne sont pas adoptés en raison de problèmes d'intégration et de passage à l'échelle.

Dans ce contexte, il devient nécessaire de proposer une plateforme générique, scalable et extensible. Cette thèse introduit **KIMVIEWare**, une plateforme basée sur les microservices pour la génération automatique de tests logiciels.

Notre projet contribue à cette vision en étendant la Phase 1 de KIMVIEWare par l'ajout du support du langage JavaScript via l'outil d'exécution symbolique **Arcon**.

## 1.2 Énoncé du problème

Les approches traditionnelles de génération de tests souffrent d'inefficacité, de redondance et d'une faible capacité de passage à l'échelle. L'exécution symbolique, bien que précise, est limitée par l'explosion des chemins.

Les algorithmes génétiques permettent d'optimiser les suites de tests, mais restent souvent appliqués à des contextes isolés. Par ailleurs, leur intégration dans des plateformes industrielles reste limitée.

Ces limitations mettent en évidence le besoin d'une plateforme générique, modulaire et scalable, capable de combiner efficacement les techniques symboliques et d'optimisation.

## Représentation mathématique du problème

Soit un programme  $P$  modélisé par un graphe de flot de contrôle (CFG)  $G = (N, E)$ , où  $N$  est l'ensemble des nœuds et  $E \subseteq N \times N$  l'ensemble des arcs.

Un chemin d'exécution est une séquence :

$$\pi = (n_1, n_2, \dots, n_k)$$

telle que  $(n_i, n_{i+1}) \in E$ .

Les décisions de branchement sont représentées par des variables booléennes  $d_j \in \{T, F\}$ , et  $\Pi$  désigne l'ensemble des chemins possibles.

Si le programme contient  $n_b$  branches binaires indépendantes :

$$|\Pi| = 2^{n_b}$$

Pour une boucle  $\ell$  avec  $k_\ell$  itérations et  $m_\ell$  tests binaires :

$$|\Pi_\ell| = (2^{m_\ell})^{k_\ell}$$

Pour un programme général :

$$|\Pi| = 2^{n_b} \cdot \prod_{\ell=1}^L (2^{m_\ell})^{k_\ell}$$

Cette expression montre la croissance exponentielle du nombre de chemins, rendant toute exploration exhaustive irréaliste.

## Conséquences pratiques

Cette explosion combinatoire entraîne plusieurs difficultés :

- **Complexité des contraintes** : résolution de contraintes distribuées.
- **Coût d'exécution élevé** : déploiement de multiples services.
- **Redondance des tests** : gaspillage de ressources.
- **Non-déterminisme** : tests instables dus aux effets réseau.
- **Limites industrielles** : difficulté d'intégration dans les pipelines CI/CD.

Ces défis expliquent les limites des approches actuelles dans les systèmes multi-services.

Une solution efficace doit :

- Composer les contraintes de manière distribuée
- Réduire l'espace de recherche
- Optimiser les suites de tests (ex. algorithmes génétiques)
- Assurer une orchestration scalable

KIMVIEWWare s'inscrit précisément dans cette perspective.

## 2 État de l’art – Concepts clés

### 2.0.1 Test logiciel (Software Testing)

Le test logiciel consiste à exécuter un programme afin de détecter des défauts et vérifier le respect des exigences. On distingue deux grandes familles :

- **Test structurel (boîte blanche)** : basé sur la structure interne du code (chemins, branches). Il est précis mais coûteux.
- **Test fonctionnel (boîte noire)** : basé uniquement sur les spécifications. Il est particulièrement adapté aux systèmes où le code n’est pas accessible.

La génération automatique de tests permet de produire rapidement de grandes suites de tests, mais elle soulève plusieurs défis : détermination du nombre de tests nécessaires, difficulté du problème de l’oracle, et gestion des cas extrêmes.

### 2.0.2 Génération de cas de tests

La génération de cas de tests est le processus de création systématique d’entrées et de sorties attendues pour évaluer un programme. Elle se déroule en plusieurs étapes :

1. Identification du domaine d’entrée
2. Modélisation des contraintes
3. Choix de la stratégie de génération (aléatoire, symbolique, par recherche)
4. Exécution des tests
5. Évaluation de l’adéquation (couverture, score de mutation)

Le problème de l’oracle correspond à la difficulté de déterminer si un résultat est correct. Des solutions incluent le test métamorphique, le test de mutation et les oracles approximatifs.

### 2.0.3 Algorithmes génétiques (GA)

Les algorithmes génétiques sont inspirés de la sélection naturelle et permettent de faire évoluer une population de solutions candidates. Leur fonctionnement repose sur les étapes suivantes :

1. Initialisation
2. Évaluation de la fitness
3. Sélection des parents
4. Croisement
5. Mutation
6. Terminaison

Ils sont utilisés, par exemple, pour générer des séquences d’appels API afin de maximiser la couverture des chemins.

**Avantages** : exploration efficace d’espaces complexes.

**Inconvénients** : réglage délicat et coût computationnel élevé.

#### 2.0.4 Exécution symbolique

L'exécution symbolique remplace les entrées concrètes par des variables symboliques. Chaque branchement du programme engendre une duplication de l'état :

$$\pi \leftarrow \pi \wedge \text{condition}$$

$$\pi \leftarrow \pi \wedge \neg \text{condition}$$

Un solveur SMT (ex. Z3) est utilisé pour vérifier la satisfiabilité des conditions de chemin.

Les principales composantes sont :

- l'état symbolique ( $\sigma$ ),
- le compteur ordinal ( $pc$ ),
- la condition de chemin ( $\pi$ ).

Cette technique est limitée par l'explosion des chemins et la complexité des solveurs.

#### 2.0.5 Architecture microservices

L'architecture microservices consiste à décomposer un système en services indépendants communiquant via des API légères.

Ses principales caractéristiques sont :

- indépendance des services,
- granularité métier,
- orchestration (ex. Kubernetes),
- tolérance aux pannes.

Dans le contexte du test logiciel, elle offre scalabilité et modularité, mais introduit des défis liés aux interactions distribuées.

#### 2.0.6 Patrons de conception (Design Patterns)

Les patrons de conception sont des solutions réutilisables à des problèmes récurrents en génie logiciel.

| Famille         | Exemples                | Utilité dans KIMVIEWare                                                                                   |
|-----------------|-------------------------|-----------------------------------------------------------------------------------------------------------|
| Créationnels    | Factory, Singleton      | Instanciation dynamique des extracteurs symboliques selon le langage.                                     |
| Structurels     | Bridge, Adapter, Facade | Découplage entre orchestration et implémentation. Permet l'intégration d'Arcon sans modification du cœur. |
| Comportementaux | Strategy, Observer      | Adaptation dynamique des algorithmes et suivi des métriques.                                              |

TABLE 1 – Utilisation des design patterns dans KIMVIEWare

Dans notre projet, le pattern **Bridge** est utilisé pour intégrer l'extracteur JavaScript (Arcon) tout en maintenant l'indépendance du cœur du système.

## 3 Workflow, pipeline et architecture microservices

### 3.1 Workflow global (Pipeline général)

KIMVIEWWare est organisé en cinq phases séquentielles mais asynchrones, orchestrées par un *message broker* (RabbitMQ). Chaque phase est implémentée comme un microservice indépendant, déployable et scalable séparément. Le workflow transforme un projet source (SUT) en un jeu de tests optimisé, prêt à être exécuté dans un pipeline CI/CD.

#### 3.1.1 Vue d'ensemble du pipeline

[CI/CD trigger]

[PHASE 0 : VALIDATION] → [PHASE 1 : EXTRACTION] → [PHASE 2 : RÉDUCTION]  
→ [PHASE 3 : OPTIMISATION] → [PHASE 4 : EXÉCUTION + FEEDBACK]

#### 3.1.2 Description détaillée des phases

**Phase 0 : Validation et détection du langage**    **Microservice : Validator**

**File d'entrée :** `job.upload` (dépôt compressé ou URL Git)

**File de sortie :** `validation.completed`

**Traitements :**

- Décompression du projet et vérification de l'intégrité (intégrité des archives, présence des fichiers requis).
- Détection heuristique du langage de programmation selon les règles suivantes (par ordre de priorité) :
  1. Présence de `package.json` → JavaScript/TypeScript.
  2. Présence de `tsconfig.json` → TypeScript.
  3. Présence de fichiers `.js` ou `.jsx` (hors `node_modules`) → JavaScript.
  4. Présence de fichiers `.ts` ou `.tsx` → TypeScript.
  5. Fichiers `.py` → Python ; `.c/.cpp` → C/C++ ; `.java` → Java.
- Création d'un identifiant unique (`job_id`) pour la traçabilité.
- Stockage des métadonnées (langue, chemin extrait, horodatage) dans la base de données (MongoDB).

**Sortie :** message JSON contenant `job_id`, `sut_info` (nom, langue, point d'entrée détecté) et `extracted_path`. Ce message est publié sur la file `validation.completed`. En cas d'échec (langue non supportée, archive corrompue), un message d'erreur est dirigé vers une file dédiée (`validation.failed`).

**Phase 1 : Extraction symbolique des chemins**    **Microservice : PathExtractorWorker**

**File d'entrée :** `validation.completed`

**File de sortie :** `extraction.completed`

**Traitements :**

- Lecture du message et récupération du chemin du projet extrait ainsi que de la langue détectée.

- **Sélection dynamique de l'extracteur** conforme au principe Open/Closed (OCP) : le service utilise un registre d'extracteurs. Selon la langue, il instancie la classe adéquate sans aucune structure conditionnelle dans le code de routage.
- Les extracteurs implémentés sont :
  - **PythonExtractor** : analyse statique via le module `ast` de Python, construction du CFG, puis exécution symbolique avec Angr (intégration de Z3 pour la résolution de contraintes).
  - **CExtractor / CppExtractor** : utilisation de Clang pour générer le LLVM IR, puis exécution symbolique via KLEE (avec solveur STP ou Z3).
  - **JavaExtractor** : analyse du bytecode avec `javalang` et construction d'un CFG, exploration symbolique via une adaptation de JBSE.
  - **JSEExtractor** (notre contribution) : spawn d'un processus Node.js exécutant le parseur Acorn (ECMAScript 2022). L'AST JSON est récupéré, puis un CFG est construit en mémoire. Un parcours en profondeur (DFS) des nœuds de branchement (if, for, switch, while, ternary) génère les trajectoires symboliques. Chaque trajectoire est validée par Z3.
- **Génération des trajectoires** : pour chaque chemin syntaxiquement possible, on extrait la condition de chemin  $\Pi(t)$  et on interroge le solveur SMT. Seules les trajectoires satisfiables (SAT) sont conservées.
- Respect des budgets : timeout global  $\Delta_{\text{ext}}$  (ex. 30 minutes) et timeout par requête SMT  $\tau_{\text{SMT}}$  (ex. 5 secondes).
- Les trajectoires sont enrichies de métadonnées : identifiant, blocs de base, condition de chemin, branches couvertes, coût estimé, etc.

**Sortie** : ensemble  $T = \{\text{trajectoires réalisables}\}$  sérialisé en JSON, publié sur la file `extraction.completed`. Le nombre de trajectoires extraites est typiquement de l'ordre de quelques centaines à quelques milliers selon la taille du service.

## Phase 2 : Réduction SGATS (Selection-Guided Aggregation of Trajectories Symbolic) Microservice : PathReductionWorker

**File d'entrée** : `extraction.completed`

**File de sortie** : `reduction.completed`

**Traitements** :

- Réduction de l'ensemble  $T$  initial (souvent volumineux et redondant) à un sous-ensemble compact  $T_{\text{reduced}}$  tout en préservant 100% de la couverture des branches.
- **Calcul de priorité**  $\rho(t)$  pour chaque trajectoire, combinant :
  - Gain marginal de couverture  $\text{cov}(t) = \frac{|Br(t) \setminus B_{\text{seen}}|}{|B|}$ ,
  - Coût estimé  $\text{cost}(t)$  (fonction des appels SMT et de la longueur du chemin),
  - Unicité structurelle  $\text{Uniqueness}(t) = 1 - \max_{t' \in T_{\text{reduced}}} \text{sim}(t, t')$ .
- **Fonction de similarité**  $\text{sim}(t_i, t_j)$  :
  - Composante structurelle : préfixe commun le plus long (LCP) normalisé,
  - Composante sémantique : distance de Hamming sur les décisions booléennes.
- **Fusion (fusion) des chemins** : lorsque deux trajectoires sont jugées similaires (seuil  $\tau$  et  $p$ ), elles sont fusionnées en une seule qui conserve l'union des branches couvertes.
- **Détection de Fusion Hit** : mécanisme de subsomption dynamique qui identifie deux trajectoires arrivant dans le même état symbolique à un même nœud. La trajectoire redondante est tronquée et ne poursuit pas l'exploration.

- L'algorithme est en temps polynomial  $O(N^2 \cdot L)$  et garantit théoriquement  $B_T \subseteq B_R$  (théorème de préservation de la couverture).

**Sortie :**  $T_{\text{reduced}}$  dont la taille est typiquement réduite de 85% (observé sur les benchmarks). Les trajectoires conservées sont celles ayant la plus haute valeur ajoutée pour la couverture et la diversité.

### Phase 3 : Optimisation EvoPath-GA (Evolutionary Path Genetic Algorithm)

**Microservice :** PathOptimizerWorker

**File d'entrée :** reduction.completed

**File de sortie :** optimization.completed

**Traitements :**

- Le problème est formalisé comme un problème de sélection de sous-ensemble (Set Cover) : trouver  $S \subseteq T_{\text{reduced}}$  qui maximise une fonction de fitness multi-objectif.
- **Encodage des chromosomes :** vecteur binaire de longueur  $|T_{\text{reduced}}|$ , chaque bit indiquant si la trajectoire  $t_i$  est incluse dans la suite de tests candidate.
- **Fonction de fitness**  $\mathcal{F}(C) = w_{\text{cov}} \cdot f_{\text{cov}}(C) + w_{\text{div}} \cdot f_{\text{div}}(C) + w_{\mu} \cdot f_{\mu}(C) - w_{\text{cost}} \cdot f_{\text{cost}}(C)$ .
  - $f_{\text{cov}}$  : proportion de branches couvertes par  $S$ .
  - $f_{\text{div}}$  : diversité de  $S$  par rapport au reste de la population (inverse de la similarité moyenne de Jaccard).
  - $f_{\mu}$  : historique du score de mutation (transmis via la boucle de rétroaction).
  - $f_{\text{cost}}$  : coût normalisé de  $S$  (pénalité).
- **Opérateurs génétiques :**
  - Sélection par tournoi (taille 3).
  - Croisement en deux points.
  - Mutation bit-flip avec probabilité  $p_m = 0.01$ .
- **Mécanismes avancés :**
  - Élitisme : les meilleurs  $e$  individus sont préservés d'une génération à l'autre.
  - Anti-clonage dynamique : les chromosomes trop similaires (seuil  $\delta(g)$ ) sont fusionnés ou éliminés pour maintenir la diversité.
  - Injection heuristique : la population initiale est enrichie de solutions construites par glouton (couverture) et par priorité SGATS.
- **Boucle de rétroaction :** les poids  $w_{\text{cov}}, w_{\text{div}}, w_{\mu}, w_{\text{cost}}$  sont ajustés périodiquement en fonction des métriques réelles ( $\mu, \kappa_{\text{act}}$ ) remontées par la Phase 4.

**Sortie :** le chromosome de meilleure fitness, qui représente le jeu de tests optimal  $S$ . Ce jeu est sérialisé et publié sur la file optimization.completed.

### Phase 4 : Exécution, collecte des métriques et rétroaction

**Microservice :** ExecutionWorker

**File d'entrée :** optimization.completed

**File de sortie :** feedback.ga (métriques vers la Phase 3)

**Traitements :**

- **Concrétisation des chemins :** pour chaque trajectoire  $t$  du jeu  $S$ , on utilise le solveur SMT (Z3) pour générer des entrées concrètes satisfaisant la condition de chemin  $\Pi(t)$ .
- **Exécution des tests :** les entrées sont fournies au microservice cible dans un environnement conteneurisé (Docker) isolé. On collecte les sorties, les traces d'exécution et les éventuelles erreurs.



- **Calcul du score de mutation  $\mu$**  :
  - Des mutants sont injectés dans le code source (via des outils comme MutPy pour Python, Mull pour C++, etc.) ou directement au niveau du bytecode.
  - Le jeu de tests  $S$  est exécuté sur chaque mutant.
  - $\mu = \frac{\text{nombre de mutants tués}}{\text{nombre de mutants non équivalents}}$ .
- **Mesure du coût réel  $\kappa_{\text{act}}$**  : temps CPU, mémoire consommée, durée d'exécution, appels réseau. Ces valeurs servent à calibrer le modèle de coût utilisé dans la fitness.
- **Stockage des artefacts** : chaque exécution produit un artefact  $\mathcal{A}(s_i)$  contenant les logs, la couverture, le temps, le coût, la version du SUT, etc. Ces artefacts sont stockés dans MongoDB (métadonnées) et MinIO (fichiers lourds).
- **Boucle de rétroaction** : les métriques ( $\mu$ ,  $\kappa_{\text{act}}$ , couverture réelle) sont envoyées sur la file `feedback.ga`. Le `PathOptimizerWorker` les consomme pour ajuster les poids  $w_i$  de la fonction de fitness. Par exemple, si  $\mu$  est inférieur à 90%, le poids  $w_\mu$  est augmenté temporairement pour favoriser les chemins qui tuent plus de mutants.

**Sortie** : les métriques sont également exposées sur un tableau de bord (Prometheus/Grafana) sous forme de KPI industriels : Mutation Score, Cost Reduction Ratio, Time-to-Coverage. Ces indicateurs permettent de décider si le jeu de tests peut être accepté dans un pipeline CI/CD (ex. fusion bloquée si  $\mu < 90\%$ ).

**Boucle de rétroaction (Feedback Loop) Mécanisme** : asynchrone, via la file `feedback.ga` et l'abonnement du `PathOptimizerWorker`.

**Objectif** : transformer KIMVIEWWare en un système auto-adaptatif. Les métriques réelles (coût d'exécution, score de mutation) sont utilisées pour corriger les estimations théoriques de la Phase 3. Ainsi, les générations futures de l'algorithme génétique tiennent compte de l'environnement réel d'exécution.

**Impacts observés** : après quelques cycles de rétroaction, le score de mutation atteint en moyenne 92,4% (contre 90,1% pour l'exécution exhaustive) et le coût est réduit de 86,3%, validant l'intérêt de cette boucle opérationnelle.

Ceci clôt la description détaillée des phases.

### 3.1.3 Garanties du pipeline

- **Terminaison** : budgets temporels bornés
- **Préservation de la couverture** :  $B_T \subseteq B_R$
- **Répétabilité** : graines GA enregistrées
- **Scalabilité** : réplication des workers (Kubernetes)

### 3.1.4 Intégration CI/CD

Le pipeline est déclenché automatiquement via webhook. Les métriques sont exposées via Prometheus et visualisées avec Grafana.

- Score de mutation ( $\geq 90\%$ )
- Taux de réduction ( $\geq 80\%$ )
- Coût d'exécution
- Time-to-Coverage (TTC)

Ces indicateurs permettent de contrôler la qualité et d'intégrer le test dans un processus industriel continu.

## 3.2 Architecture microservices

L'architecture est structurée en trois couches principales.

### 3.2.1 Couche de communication

RabbitMQ assure :

- la gestion des files de messages,
- la persistance,
- le découplage entre services.

Files principales :

- validation.completed
- extract.path
- reduce.path
- exec.job
- feedback.ga

### 3.2.2 Couche de traitement

Les workers sont des services *stateless*, conteneurisés (Docker) et orchestrés (Kubernetes). Ils peuvent être répliqués pour traiter les charges en parallèle.

### 3.2.3 Couche de stockage

- **MongoDB** : stockage des métadonnées et résultats
- **MinIO** : stockage des artefacts lourds
- **Prometheus** : collecte des métriques

### 3.2.4 Diagramme d'architecture

[CI/CD webhook]

Validator

RabbitMQ Broker

PathExtractor PathReduction PathOptimizer

ExecutionWorker

feedback.ga

| Pattern        | Famille        | Application                                                           |
|----------------|----------------|-----------------------------------------------------------------------|
| Bridge         | Structurel     | Découple l'orchestration des extracteurs concrets (KLEE, Angr, Arcon) |
| Decorator      | Structurel     | Ajout dynamique de fonctionnalités (instrumentation, logging)         |
| Factory Method | Créationnel    | Instanciation des extracteurs selon le langage                        |
| Observer       | Comportemental | Mise à jour du dashboard en temps réel                                |
| Strategy       | Comportemental | Changement dynamique de l'algorithme d'optimisation                   |

TABLE 2 – Patterns utilisés dans KIMVIEWare

### 3.3 Patterns de conception(il nous faut mettre un diagramme)

### 3.4 Garanties de l'architecture

- **Découplage** : communication uniquement via messages
- **Scalabilité** : réplication des workers (HPA Kubernetes)
- **Résilience** : persistance des messages en cas de panne
- **Extensibilité** : ajout de nouveaux langages via le pattern Bridge

## 4 Nos apports

### 4.1 Ingénierie logicielle et refactoring

Bien que l'architecture microservices soit robuste, le code interne nécessitait une restructuration pour garantir évolutivité et maintenabilité. Notre première contribution consiste en un refactoring basé sur les principes SOLID et les Design Patterns.

#### 4.1.1 Open/Closed Principle (OCP) – Extension sans modification

Dans le système initial, la gestion de l'analyse multi-langages (Phase 1) était fortement couplée. L'ajout d'un nouveau langage nécessitait de modifier le cœur du service de routage, ce qui violait le principe **Open/Closed** (ouvert à l'extension, fermé à la modification).

Nous avons refactorisé cette partie pour la rendre conforme à l'OCP. Désormais, le composant principal (**ExtractorService**) ne contient plus aucune condition explicite sur les langages. Il s'appuie sur un **mécanisme d'enregistrement dynamique** : chaque extracteur (Python, C, Java, JavaScript) s'enregistre auprès du service en fournissant son nom et sa logique d'extraction. Lorsqu'un nouveau langage doit être supporté, il suffit d'implémenter une nouvelle classe d'extracteur et de l'enregistrer, **sans toucher au code existant** du service.

**Avant (non conforme OCP)** : une longue structure conditionnelle

```
if language == "python": ...
elif language == "c": ...
```

```
elif language == "java": ...  
...
```

qu'il fallait modifier à chaque ajout.

**Après (conforme OCP)** : le service expose un registre (dictionnaire ou table de correspondance). L'initialisation enregistre les extracteurs disponibles. Le traitement du message ne contient plus aucun `if` sur le langage; il récupère simplement l'extracteur depuis le registre et l'exécute.

Résultat : le système est devenu **ouvert à l'extension** (prêt à recevoir notre extracteur JavaScript/TypeScript) mais **fermé à la modification**, garantissant ainsi sa stabilité lors des mises à jour. Cette amélioration a été réalisée sans changer l'interface publique du microservice, ni impacter les autres phases du pipeline.

## 4.2 Détection automatique du langage JavaScript/TypeScript (Phase 0 – Validator)

Avant notre contribution, le système ne reconnaissait pas les projets JavaScript. La Phase 0 (Validator) ne supportait que Java, Python et C/C++. Nous avons introduit une logique de détection heuristique basée sur l'analyse de la structure du dépôt.

Le langage détecté est ensuite propagé via le champ `sut_info.language` vers la Phase 1 (*ExtractorService*).

### 4.2.1 Heuristiques de détection

Le Validator applique les règles suivantes, par ordre de priorité :

- Présence de `package.json`  $\Rightarrow$  projet JavaScript/Node.js
- Présence de `tsconfig.json`  $\Rightarrow$  TypeScript
- Présence de fichiers `.js` ou `.jsx` (hors `node_modules`)  $\Rightarrow$  JavaScript
- Présence de fichiers `.ts` ou `.tsx`  $\Rightarrow$  TypeScript

Si aucune condition n'est satisfaite, le langage est défini comme `unknown` et le job est rejeté.

### 4.2.2 Algorithme de détection

ALGORITHME DetectLanguage

Entrée : chemin du projet

Sortie : langage ("javascript", "typescript", "unknown")

1. SI "package.json" existe :
2.     SI "tsconfig.json" existe :
3.         retourner "typescript"
4.     SINON :
5.         retourner "javascript"
  
6. fichiers\_js  $\leftarrow$  \*.js, \*.jsx (hors node\_modules)
7. fichiers\_ts  $\leftarrow$  \*.ts, \*.tsx (hors node\_modules)

```
8. SI fichiers_ts non vides :
9.     retourner "typescript"
10. SINON SI fichiers_js non vides :
11.     retourner "javascript"
12. SINON :
13.     retourner "unknown"
```

### 4.2.3 Intégration dans le workflow

Une fois le langage détecté, le Validator publie un message sur la file `validation.completed`. Ce message est ensuite consommé par la Phase 1.

Listing 1 – Message publié après validation

```
{
  "job_id": "job_42",
  "status": "validated",
  "sut_info": {
    "name": "mon-service-web",
    "language": "javascript",
    "entry_point": "src/index.js"
  },
  "extracted_path": "/tmp/kimvie/job_42"
}
```

Grâce au pattern *Strategy*, aucune modification du cœur du système n'a été nécessaire : le langage est simplement utilisé pour sélectionner dynamiquement l'extracteur approprié.

### 4.2.4 Robustesse et limites

Cette approche couvre la majorité des projets JavaScript/TypeScript réels :

- Détection fiable même sans `package.json` (scripts simples)
- Faible taux de faux positifs grâce aux critères combinés
- Possibilité de surcharge via un fichier `.kimvie.yml`

Ce mécanisme garantit une intégration fluide avec les phases suivantes du pipeline.

## 4.3 Extracteur JavaScript / TypeScript

Notre seconde contribution est l'ajout d'un extracteur pour les applications JavaScript/TypeScript.

### 4.3.1 Principe de fonctionnement

L'extracteur s'appuie sur un sous-processus Node.js exécutant la bibliothèque *Acorn* afin de générer un AST au format JSON.

### 4.3.2 Architecture (Pattern Bridge)(on doit rajouter un diagramme de ce pattern)

Le pattern Bridge permet de découpler l'interface commune de l'implémentation spécifique :

ExtractorService

ExtractorBase

Python C JSExtractor

Node.js + Acorn

### 4.3.3 Algorithme d'extraction

ALGORITHME JS\_symbolic\_extraction

Entrée : service JS

Sortie : ensemble de trajectoires T

1. Exécuter Acorn (Node.js)
2. Récupérer AST JSON
3. Construire CFG
4. Initialiser Q avec noeud initial
5.  $T \leftarrow$
6. Tant que Q non vide :
  7.  $t \leftarrow \text{extraire}(Q)$
  8.  $\leftarrow$  condition de chemin
  9. Si SAT() :
    10. ajouter t à T
    11. ajouter successeurs à Q
  12. Sinon :
  13. ignorer t
14. Retourner T

## 4.4 Bilan des apports

Ces contributions ont été intégrées dans la branche principale de KIMVIEWWare et sont utilisées par les phases suivantes du pipeline.

| <b>Contribution</b>              | <b>Détail technique</b>                                        | <b>Impact</b>                                                            |
|----------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------|
| Refactoring OCP + Strategy       | Interface ExtractorBase, dispatch dynamique                    | Extension sans modification du code                                      |
| Template Method                  | Classe MicroserviceBase                                        | Standardisation et réduction de la redondance                            |
| Détection automatique du langage | Heuristiques (package.json, tsconfig.json, extensions .js/.ts) | Support des projets JavaScript/TypeScript dès la Phase 0                 |
| Extracteur JS/TS                 | Node.js + Acorn + CFG                                          | Extension aux applications web et génération de trajectoires symboliques |
| Pattern Bridge                   | Découplage architecture                                        | Remplacement facile du moteur d'analyse JS                               |

TABLE 3 – Synthèse des contributions